

Why LLM Aggregation Falls Short: Correlated Errors and Selection Gaps in Arabic Punctuation Restoration

Rana Alshaikh

Department of Information Systems
Faculty of Computing and Information
Technology, King Abdulaziz University
Rabigh, 21589, Saudi Arabia
raalshaikh@kau.edu.sa

Majedah Alrehaily

Computer Science and Information
Program Unit, Branch of the Applied
College in Al-Ula, Taibah University
Madinah, Saudi Arabia
mmrehaily@taibahu.edu.sa

Abstract

Combining LLMs for Arabic punctuation restoration should help when their errors are complementary, but aggregation failure can reflect either missing coverage, where no predictor proposes the correct label, or failed selection, where the correct label is present but not selected. We investigate this distinction with vote-based, text-blind aggregators that use predictor labels, agreement, position, and neighbouring predictions, but not Arabic words. We evaluate three frontier LLMs and a fine-tuned AraBERT encoder on AraPunc-sub and ABC. The LLMs show strongly correlated errors: when all three are wrong, they choose the same incorrect label in 79.4% of cases on AraPunc-sub and 77.8% on ABC. Yet the four-predictor oracle achieves macro- F_1 scores of 0.857 and 0.778, surpassing the best standalone predictors (0.639 and 0.496). Leave-one-predictor-out analysis shows that AraBERT supplies the largest unique oracle coverage. Standard aggregation recovers only part of this headroom, and no tested aggregator outperforms standalone AraBERT on AraPunc-sub. A selectability probe confirms that some covered disagreements are learnable, but a deployable router offers only modest, corpus-dependent gains. Even a text-aware peer-deliberation step deepens rather than breaks the correlation, as weaker models conform to their peers. Hence, in the tested text-blind regime, the bottleneck is primarily selection rather than coverage, without ruling out text-aware selectors.

1 Introduction

Punctuation restoration is the process of restoring missing punctuation to improve text readability. The Arabic text can be ambiguous when diacritics are absent, since a single written form can correspond to multiple analyses; as a result, punctuation and context play an important role in text disambiguation (Habash, 2010). Arabic punctuation restoration (APR) can be more challenging as

the punctuation rules are applied differently, and these differences become more apparent across genres (Yagi et al., 2024b). The use of frontier large language models (LLMs) in a zero-shot setting for punctuation restoration showed competitive performance (Al Wazrah et al., 2025; Suen et al., 2025); therefore, it is logical to study the performance of combining them as a follow-up step. Model aggregation may reduce their errors if they are partly independent, and yield a system better than each individual model.

This paper asks whether that expectation holds and finds a more qualified answer than a simple ensemble win or loss. For token-level APR, where aggregation helps, it is unclear whether LLM errors are sufficiently complementary. Moreover, it is indefinite whether the predictor pool lacks a correct alternative (a *coverage problem*) or contains one that the aggregator fails to select (a *selection problem*) when an ensemble underperforms. These diagnoses have different engineering implications: the former calls for better predictors, while the latter calls for better selection. We therefore evaluate three frontier LLMs on two independently sourced Arabic corpora, add a structurally different encoder. Oracle and disagreement analyses were also used to distinguish missing coverage from failed selection. In this case, the oracle is non-deployable: the headroom available is estimated when the correct candidate label is present. Text-blind aggregation serves as a diagnostic control as it is not able to see the Arabic tokens, and improvements must come from the candidate-label structure alone. The results are mixed rather than uniformly negative: LLM-only voting adds little, global stackers recover only limited four-predictor oracle headroom, and disagreement-specialized selection helps in some cases but not consistently.

Our investigation is organized around four questions. (Q1) How correlated are the LLMs, and how

much oracle coverage does the committee contain? We measure inter-model error correlation directly, including what the models do when jointly wrong, and bound the committee’s coverage with a per-slot oracle. **(Q2)** Does a structurally different predictor add coverage? We fine-tune an Arabic encoder **AraBERT** (Antoun et al., 2020) and use a leave-one-predictor-out oracle to measure each predictor’s unique contribution. **(Q3)** Do standard aggregators (per-mark co-vote calibration, majority voting, class-balanced logistic and XGBoost stacking, with and without encoder features) improve over the strongest predictor available to them? **(Q4)** If standard aggregation leaves some coverage unused, can a selector recover it? We train a disagreement-specialized selector as a diagnostic. These questions mainly test whether aggregation fails because of a lack of candidate coverage or because it fails to select from available candidates. The results in the tested text-blind regime support the latter diagnosis. The pool often contains the correct label, but the tested aggregators recover only part of that headroom.

The contribution is diagnostic: we separate missing coverage from failed selection and show that, in this vote-based, text-blind setting, the main bottleneck is selection. We do not rule out input-aware selectors, few-shot prompting, fine-tuned LLMs, or logit-level committees.

2 Related Work

Punctuation restoration and Arabic. Automatic punctuation restoration has been studied as it directly affects readability, downstream parsing, and the usability of automatic speech recognition (ASR) output; foundational statistical approaches established it as a crucial step (Beeferman et al., 1998; Gravano et al., 2009). Neural sequence-labelling models subsequently improved the baseline on spoken-language transcripts (Tilk and Alumäe, 2015; Tilk and Alumäe, 2016; Courtland et al., 2020). AraPunc (Sakr and Torki, 2023) extended the task to Arabic with a token-level dataset and benchmark, on which a multilingual encoder reaches strong in-domain performance. Beyond fine-tuned encoders, Al Wazrah et al. (Al Wazrah et al., 2025) report LLMs competitive with AraBERT but non-dominant across marks. Across the extant literature, the main objective has been to build stronger predictors that work *individually*. However, the question of what hap-

pens when several predictors operate simultaneously and whether combining them could outperform the best predictor individually has received little attention. This distinction is important as the benefit of the ensemble depends not only on the quality of each predictor but also on whether their errors are complementary. We therefore evaluate ensemble behaviour across two Arabic corpora and include AraBERT (Antoun et al., 2020) as a structurally different, fine-tuned token classifier.

Ensemble Methods and coverage-selection gap.

Classical ensemble theory states that the benefit of combining models depends on the diversity of their errors rather than on ensemble size or the quality of individual models. Previous research shows that committees improve only when member errors are decorrelated (Hansen and Salamon, 1990), diversity is the main constraint (Dietterich, 2000), high covariance between predictors reduces the gain (Krogh and Vedelsby, 1994), and stacking (Wolpert, 1992) has this limitation when correlating base predictions (Kuncheva and Whitaker, 2003). Recent LLM ensemble methods inherit this requirement but rarely test it. Self-consistency (Wang et al., 2023) and fusion methods such as LLM-Blender (Jiang et al., 2023) degrade on homogeneous pools. Model-difference studies (Wan et al., 2024) restate diversity in LLM terms. Multi-agent debate (Du et al., 2024; Liang et al., 2024) presupposes genuinely distinct beliefs rather than convergent priors from shared pretraining. Routing methods (Lu et al., 2024; Ong et al., 2025) avoid combination entirely by selecting a single model for each query, assuming the best output can be identified directly. What combination can achieve is bound by oracle analyses across ASR (Fiscus, 1997), machine translation (Matusov et al., 2006; Rosti et al., 2007), parsing (McClosky et al., 2006), and QA (Min et al., 2021). They consistently show that the correct answer is present in the pool far more often than voting recovers, but rarely separate whether this comes from missing coverage or failed selection. The complementary signal may be absent (a coverage problem) or present but not selected (a selection problem). Prior work often conflates these two cases, even though they require opposite solutions.

3 Methodology

3.1 Task Formulation

Punctuation restoration is formulated as a token-level sequence labelling task. Given an unpunctuated paragraph of word tokens $x_1, \dots, x_N$, each token receives a label from $\mathcal{M} \cup \{O\}$, where $\mathcal{M} = \{\text{COMMA}, \text{SEMICOLON}, \text{PERIOD}, \text{COLON}, \text{QUESTION}, \text{EXCL}\}$ denotes the punctuation mark that follows the token and O indicates no punctuation. The primary evaluation metric is *macro-averaged* F_1 across the classes in the test split. This is important for AraPunc-sub, whose test split contains no exclamation marks. Because most tokens are labeled O , token-level accuracy may be misleading. Therefore, *micro- F_1* over non- O tokens is also reported for individual predictors as a descriptive secondary analysis. *Macro- F_1* serves as the primary metric for all system comparisons, bootstrap tests, and oracle analyses, while *micro- F_1* is used solely to characterize individual predictors and is not used for comparative evaluation.

We define *coverage* as the percentage of slots (token positions where a mark may appear), in which at least one predictor proposes the correct (gold) label and the labels proposed for a slot form its *candidate set*. Instead of asking whether a deployable system can identify the correct label, coverage asks only whether the correct label is available somewhere in the candidate set proposed by the predictors. We use a *per-slot oracle* (also called a *candidate-set oracle*) as a non-deployable diagnostic, since it chooses the gold label when the candidate set contains it and is forced to be wrong otherwise; as a result, we report its macro- F_1 as *oracle macro- F_1* .

Two predictor pools are used across this study. The *LLM-only pool*, which contains the three LLMs. The *four-predictor pool*, which contains the three LLMs plus AraBERT. Two oracles are computed: the *LLM-only oracle* over the first pool, and the *four-predictor oracle* over the second. The drop in four-predictor oracle macro- F_1 when that predictor is removed is the predictor’s *unique contribution*.

3.2 Corpora

Our evaluation includes two independently sourced Arabic corpora (full statistics in Appendix B, Tables 9–10). **AraPunc-sub** is a deterministic, cost-bounded sub-sample of the official AraPunc (Sakr and Torki, 2023) dev/test

split (segmentation in Appendix B). We use the name *AraPunc-sub* for all reported scores and reserve *AraPunc* for the official benchmark. As a sub-sample is evaluated under a support-aware macro- F_1 convention, absolute scores are not comparable with full-benchmark results. All claims are relative comparisons among systems on the same sampled units. AraPunc-sub is comma-dominant, with heavy use of colons, reflecting the long, coordinated sentences typical of its classical/MSA register. **ABC** is the manually annotated gold-standard component of the Arabic Punctuation Dataset (Yagi et al., 2024a), a formal-register corpus of non-fiction book chapters, split as 10dev / 5test documents and segmented as presented in Appendix B. ABC is period-dominant and roughly twice as densely punctuated as AraPunc-sub (17.4 vs. 9.5 marks per 100 tokens), consistent with the shorter declarative sentences of formal non-fiction.

3.3 Predictors

In the zero-shot setting at temperature 0, we instruct each of the three LLMs, **GPT-5.5**, **Claude Sonnet 4.6**, and **Gemini 2.5 Flash**, to return the input words with one punctuation label per word. We collect their predictions and apply a sequence-matching soft-alignment procedure to both corpora before evaluation to address minor differences in word count or formatting. Only segments where all three LLM outputs are parseable and align with the input are included in the evaluation.

We use a single shared prompt across all models, since our goal is to test whether the model families make complementary errors, not to optimize prompting. As a result, the correlation estimates reflect properties of this matched zero-shot committee (prompt and control justification in Appendix G, H). As a fourth predictor, we fine-tune AraBERT as a token classifier using split-disjoint training segments from the complement of each corpus’s split. Hyperparameters are selected per corpus on a held-out validation split (Appendix H). AraBERT is the only non-LLM predictor in the four-predictor pool. We use it in the oracle, complementarity, and stacking analyses (Section 4).

3.4 Aggregation and Post-Processing

We evaluate several discrete-label aggregation methods, each built around a different assumption about how predictors should be combined (full specifications in Appendix H). The first, which we

refer to as Stage A, is per-mark co-vote calibration. It learns an integer threshold $k_m \in \{1, 2, 3\}$ for each mark, and keeps a predicted mark only when at least k_m of the three LLMs agree on it. The second method, Stage B, comprises a family of per-slot multiclass stackers, including logistic regression and XGBoost. Features for these models include one-hot LLM predictions, agreement indicators, token position, and a local prediction window. Each stacker is trained with and without AraBERT features, as well as with or without the Stage A co-vote calibration (eight dev-selected configurations), from which one is selected based on performance on the development split. The third method employs AraBERT directly, both independently and as part of the predictor pool. Finally, we include parameter-free majority voting as a baseline, breaking ties toward *O*.

3.5 Diagnostic Analyses

Beyond aggregate scores, four diagnostics let us reason about *why* aggregation succeeds or fails.

Error correlation. We use three agreement measures to examine whether the three LLMs tend to make similar predictions: mean pairwise Cohen’s κ , slot unanimity, which measures how often all three LLMs agree, and the Jaccard overlap of false-comma positions across model pairs. We limit the overlap analysis to false commas because these are the dominant shared errors and the only mark with enough false positives to enable reliable estimation of overlap. The heavily dominant *O* (no-punctuation) class inflates overall κ and unanimity. To address this, we also report error-conditioned agreement: κ restricted to non-*O* and gold-non-*O* slots, alongside the rate at which all three LLMs assign the same wrong label and base the correlation claim on these.

Per-slot oracle and leave-one-out. A per-slot oracle counts a slot as covered if any predictor in a named pool emits the gold label at that slot. We report oracle coverage and oracle macro- F_1 for two pools: the LLM-only pool and the four-predictor pool. The leave-one-out analysis removes one predictor at a time from the four-predictor pool and measures the resulting drop in oracle macro- F_1 . A substantial drop indicates that the removed predictor emits the correct label at slots where the others did not. We refer to this as *complementary correct coverage* rather than error decorrelation, since we do not measure error correlation directly.

One plausible explanation is that AraBERT’s Arabic pretraining and fine-tuning make it more sensitive to local morphological cues that the zero-shot LLMs do not always convert into correct punctuation labels. Finally, we ask which punctuation marks contribute to the gap between oracle coverage and system performance by identifying *covered-but-missed* slots, where the gold label is present in the candidate set but not selected by any system.

Disagreement-focused selection (Q4). Q4 separates whether the correct label is present among the four predictions from whether it can be selected without gold labels. We test the former with a gold-label probe over *covered-disagreement* slots, where at least one predictor is correct and at least one is wrong; this is diagnostic only, asking whether the correct choice is learnable in principle. We test the latter with deployable, text-blind *router* over *contested* slots, where the four predictors do not all agree and which are identifiable without gold. The selector chooses among the candidate labels using the predictions alone, without reading the Arabic text. We also include a fixed baseline, the *trust-the-dissenting-encoder rule*, that selects the encoder when it is the sole dissenter, testing whether its complementary correct coverage from Q2, the largest among the predictors, remains useful without training a selector. Definitions are in Appendix E, and a separate text-aware peer-deliberation probe is in Appendix F.

Significance. All system comparisons use paired bootstrap resampling over evaluation segments (Koehn, 2004; Dror et al., 2018). We report the mean macro- F_1 difference together with a 95% bootstrap confidence interval (Bestgen, 2022) and a one-sided p -value, treating an effect as significant only when the interval excludes zero. Every comparison names its reference point: the best LLM, the best standalone predictor, or the best standard stacker.

4 Results

4.1 Individual Models and the Correlation Problem (Q1)

Table 1 shows that the best individual LLM, across both corpora, is modest; Gemini scores macro- F_1 0.506 on AraPunc-sub and 0.477 on ABC. The majority voting does not reliably improve performance, changing the score by at most a few

System	macro- F_1		micro- F_1 (non- O)	
	AraPunc-sub	ABC	AraPunc-sub	ABC
GPT-5.5	0.503	0.444	0.440	0.515
Claude	0.400	0.388	0.412	0.473
Gemini 2.5	0.506	0.477	0.434	0.540
Majority vote	0.510	0.461	—	—
AraBERT	0.639	0.496	0.609	0.632

Table 1: Individual predictors and majority voting, by macro- F_1 and by micro- F_1 over non- O tokens. The best LLM is bold per corpus/metric.

thousandths (+0.003 on AraPunc-sub, −0.016 on ABC). The model ordering is identical on both corpora: Gemini > GPT-5.5 > Claude. AraBERT has the best performance overall, with 0.639 on AraPunc-sub and 0.496 on ABC; full per-model macro- and micro- F_1 scores appear in Table 1. AraBERT is fine-tuned on corpus-specific labels, so its standalone scores are not resource-equivalent to the zero-shot LLMs. Therefore, it is used as the non-LLM member of the four-predictor pool, not as a like-for-like “encoder vs. LLM” comparison.

Voting fails because the models do not make independent errors. Table 2 shows mean pairwise Cohen’s κ of 0.705 on AraPunc-sub and 0.739 on ABC, with 88.7% and 89.8% of test slots receiving unanimous predictions. These values are inflated by the dominant O class; therefore, error-conditioned agreement is also reported. When at least one LLM predicts a mark, mean pairwise κ is 0.453 on AraPunc-sub and 0.500 on ABC; on gold-non- O slots, 0.617 and 0.631, respectively.

A clear evidence that the models fail *together*, is what happens when they are all wrong: among slots where all three LLMs are wrong, they assign the *same* wrong label on 79.4% of cases on AraPunc-sub ($n=2,520$ all-three-wrong slots) and 77.8% on ABC ($n=1,733$). In other words, around four in five joint failures are not three independent errors, but one shared error repeated three times. This is exactly the case where voting and agreement-based filtering do not correct the error; instead, they reinforce it. Consistent with this, when the models place a false comma, they tend to place it in the same position. False-comma overlap across model pairs reaches a Jaccard of 0.436 on AraPunc-sub and 0.457 on ABC. On ABC, that is about 19–23 $\times$ the overlap expected under independent errors. This pattern is consistent with Arabic clause structure, where long sentences are

chained by the coordinating particle *wa-* (and) and clause boundaries are often signalled by conjunction rather than by punctuation, so comma placement is genuinely underdetermined; the models default to marking coordination boundaries that the gold annotation leaves unmarked (Yagi et al., 2024b).

4.2 A Fine-Tuned Encoder Adds Complementary Coverage (Q2)

Despite the correlation, the candidate set often contains the gold label. The three LLMs cover 92.4% of test slots on AraPunc-sub and 92.2% on ABC; adding AraBERT raises overall coverage to 96.6% and 95.9%. Because the dominant O class inflates overall coverage, the more informative view is gold-non- O coverage: the four-predictor pool covers 83.3% of non- O AraPunc-sub slots and 82.9% of non- O ABC slots, compared with 73.3% and 63.0% for the three LLMs alone. Table 5 shows that AraBERT substantially increases non- O coverage on both corpora, although coverage remains limited for some uncommon marks, such as semicolons on ABC.

We use the oracle to determine whether the correct punctuation label is present anywhere in the candidate set, without considering whether a real system could select it. Oracle macro- F_1 provides the class-balanced view. It is also the basis for the presence-versus-selection gap, since it is comparable to the systems own macro- F_1 . For aggregators that must select from the candidate labels, it acts as a ceiling. For our stackers, it is a diagnostic reference, since they could, in principle, output a label that is not in the candidate set. It is not a formal mathematical upper bound for every aggregator, and it is also less clean than the binary coverage numbers, because it uses a fixed forced-wrong fallback. All oracle values reported below are oracle macro- F_1 under the forced-wrong fallback.

The three-LLM oracle macro- F_1 reaches, in Table 3, 0.690 on AraPunc-sub and 0.642 on ABC, substantially higher than the best LLM (0.506, 0.477). Adding the fine-tuned encoder raises the four-predictor oracle macro- F_1 to 0.857 and 0.778, which leaves gaps of 0.218 and 0.282 over the best individual predictors. This raises the question: which predictor supplies the additional headroom?

The answers are presented in Table 4 through the leave-one-predictor-out oracle. It removes each predictor from the four-predictor pool and measures how much oracle macro- F_1 drops. Re-

Correlation measure	AraPunc-sub	ABC
3-LLM slot unanimity	88.7%	89.8% [.884, .911]
Mean pairwise κ (all)	0.705 [.691, .719]	0.739 [.709, .766]
κ , ≥ 1 LLM non- O	0.453	0.500
κ , gold non- O	0.617	0.631
Same wrong label all 3 wrong	79.4%	77.8%
n all-3-wrong slots	2,520	1,733
False-comma Jaccard	0.436 [.409, .462]	0.457 [.418, .502]
Ratio vs. independence	16.6–18.1 $\times$	18.7–22.6 $\times$

Table 2: Inter-model error correlation on the test split. CIs are shown in brackets. The error-conditioned κ rows reduce the inflation caused by the dominant O class, and the same-wrong-label row shows that when all three LLMs fail they usually choose the same incorrect label.

Corpus	Oracle		Best real	
	3-LLM	4-pred	stand.	LLM
AraPunc-sub	0.690	0.857	0.639	0.506
ABC	0.642	0.778	0.496	0.477

Table 3: Oracle macro- F_1 (3-LLM and 4-predictor pools) vs. the best standalone predictor and best LLM.

moving AraBERT on AraPunc-sub drops the oracle by 0.089 (95% CI [.079, .099]), which is four times the contribution of any individual LLM (≤ 0.022). Removing AraBERT on ABC drops it by 0.136 (95% CI [.111, .161]), which is again the largest single contribution by a wide margin, ahead of GPT-5.5 (0.039), Gemini (0.020), and Claude (0.007). AraBERT, on both corpora, is correct on many slots where all three LLMs are wrong. We therefore describe it as providing the most *complementary correct coverage*. This does not mean that the errors are globally decorrelated, as that would require direct AraBERT–LLM correlation measures, which are outside the scope of this paper. This contribution is unlikely to reflect noise: although a weak predictor diverging arbitrarily from the LLMs could inflate apparent coverage, the standalone scores in Table 1 indicate otherwise, on ABC, AraBERT contributes the most unique coverage while also scoring 0.496 standalone, comparable to the best LLM’s 0.477. A system that is itself near the top of the pool is far from random, so its coverage is unlikely to be pure noise.

The Q2 result is narrower: AraBERT contributes more unique oracle macro- F_1 than any individual LLM on both corpora, and instead supports selecting ensemble members by both competence and error complementarity. Whether this added coverage is selectable at inference time is tested in Q3 and Q4.

Corpus	Predictor removed	Unique contrib.
AraPunc-sub	GPT-5.5	0.022 [.016, .028]
	Gemini	0.013 [.009, .019]
	Claude	0.013 [.010, .015]
	AraBERT	0.089 [.079, .099]
ABC	GPT-5.5	0.039 [.013, .067]
	Gemini	0.020 [.009, .036]
	Claude	0.007 [.005, .010]
	AraBERT	0.136 [.111, .161]

Table 4: Leave-one-predictor-out oracle: drop in four-predictor oracle macro- F_1 when each predictor is removed, with 95% bootstrap CI. Larger drops indicate greater complementary correct coverage. AraBERT is the largest unique contributor on both corpora.

	Pool	All non- O	,	.	:	;
AraPunc-sub	LLMs	.924	.733	.638	.902	.782
	4-pred	.966	.833	.788	.934	.844
ABC	LLMs	.922	.630	.602	.563	.931
	4-pred	.959	.829	.784	.853	.963

Table 5: Candidate coverage by pool, overall, on gold-non- O slots, and by mark. Adding AraBERT substantially increases non- O coverage on both corpora; question marks are omitted for space (four-predictor coverage 0.91 on both corpora).

Robustness to the oracle fallback. When the oracle is recalculated with uncovered slots supplied by the best individual predictor, instead of the forced-wrong fallback, the four-predictor oracle only slightly shifts (AraPunc-sub 0.857 $\rightarrow$ 0.847, ABC 0.778 $\rightarrow$ 0.754). The leave-one-out ordering is left unchanged: AraBERT remains the largest unique contributor by 3–4 $\times$ (Table 4).

4.3 Standard Aggregation Underuses the Coverage (Q3)

The coverage exists (Q1), and a complementary predictor supplies it (Q2), but the tested standard aggregators recover only a small part of it. Full

Corpus	System	Δ macro- F_1
AraPunc-sub	AraBERT standalone	+0.133
	Stacker (xgb, +AraBERT)	+0.063
	Majority vote	+0.003
	Stacker (logistic, raw)	-0.103
ABC	AraBERT standalone	+0.019
	Stacker (logistic, +AraBERT)	+0.036
	Majority vote	-0.016
	Stacker (logistic, raw)	-0.013
	Stacker (xgb, raw)	-0.033

Table 6: Selected aggregators vs. the best LLM (Gemini); Δ is the macro- F_1 difference. Each panel lists the systems discussed for that corpus. Significant gains on AraPunc-sub are confined to encoder-using systems, none surpassing standalone AraBERT; on ABC no interval excludes zero. Full configurations with CIs in Table 11, Appendix B.

per-system deltas against the best LLM (Gemini on both corpora) are reported in Table 6 and more details in Appendix B, C, Table 11, which show absolute scores and segment-level bootstrap intervals together. The question is whether these aggregators outperform the best predictor available to them, and we answer this by comparing them to two baselines: the best LLM and the best standalone predictor.

Aggregator versus the best LLM. No standard global aggregator significantly beats it on ABC. Majority voting is -0.016 ; the non-encoder stackers are at or below baseline (-0.013 logistic, -0.033 XGBoost); and the best encoder-feature stacker reaches only $+0.036$ (CI $[-.012, .097]$, $p=0.090$, spanning zero). On AraPunc-sub, majority voting is within noise ($+0.003$), and the only systems that significantly improve over the best LLM are those that use encoder information: AraBERT standalone ($+0.133$, CI $[.099, .165]$, $p < 0.001$) and the AraBERT-feature XGBoost stacker ($+0.063$, CI $[.041, .086]$, $p < 0.001$). LLM-only aggregation (both voting and the non-encoder stackers) does not improve on either corpus, and the non-encoder logistic stacker is significantly *worse* than the best LLM on AraPunc-sub.

Versus the best standalone predictor. This more rigorous comparison is uniformly negative on AraPunc-sub. The standalone encoder scores 0.639 , and every tested aggregation underperforms it, including the best encoder-feature stacker (0.569 , i.e. -0.070) and majority voting. None of the tested aggregators outperforms the standalone encoder on AraPunc-sub. On ABC, the encoder

is weaker (0.496), and the best encoder-feature stacker edges slightly above it in point estimate. However, the difference is not significant on five documents.

The gap between coverage and selection is large and concrete. The four-predictor oracle macro- F_1 is 0.778 on ABC (Table 3), while the best deployable standard stacker reaches 0.513 ($+0.036$ over the best LLM; Tables 6 and 11). This leaves most of the 0.27 oracle gap unrealized. The same pattern holds on AraPunc-sub against an oracle of 0.857 . We stress that this gap is a diagnostic reference, not a hard bound on the stackers. In principle, the stackers can provide a label that no predictor proposed (Table 8). The gap shows how much candidate coverage the tested selection-style aggregators leave unused. It is not an absolute ceiling they can never exceed. The natural question is whether this unused coverage can actually be selected, or whether the standard stacker fails because the signal cannot be recovered from the available features, which is addressed in Q4.

4.4 Where the Covered Errors Occur

A per-mark analysis of the covered-but-missed region (Appendix B, Table 12) shows that after normalizing by gold support, O is not the main bottleneck. Its missed-coverage rate is only 3.7% on AraPunc-sub and 2.2% on ABC. The remaining gap is concentrated in punctuation decisions: on AraPunc-sub, the main classes are semicolon, comma, and period, while on ABC, they are period, comma, and exclamation. This points to a corpus-dependent class profile rather than a single universal mark bottleneck.

4.5 Is the Unused Coverage Selectable? (Q4)

Q4 asks whether the oracle headroom left unused by standard aggregation can be recovered from the disagreement structure alone. A gold-label probe looks only at slots where the gold label is present in the candidate set but the predictors disagree. In that setting the candidate choice is partly learnable, improving by $+0.113$ macro- F_1 on AraPunc-sub and $+0.187$ on ABC over an all-slot selector (Table 7). The deployable router is more limited: it leaves unanimous four-predictor slots unchanged and sends only contested slots to a learned selector, giving only modest, corpus-dependent gains over the trust-the-dissenting-encoder rule, reaching 0.546 on AraPunc-sub and 0.536 on ABC. The gain is not significant on AraPunc-sub and signif-

Policy / probe	AraPunc-sub	ABC
<i>Gold-label probe (diagnostic)</i>		
Covered-disagreement	0.415	0.633
All slots	0.303	0.446
<i>Deployable selection</i>		
Trust-the-dissenting rule	0.535	0.476
Learned router	0.546	0.536
Δ (learned – rule)	+0.011	+0.061

Table 7: Q4 diagnostics. The gold-label probe shows the candidate choice is partly learnable (+0.112/+0.187 over an all-slot selector).

icant on ABC but rests on only five source documents; full variants are in Appendix E. Thus some unused coverage is selectable, but the tested selector does not give a robust universal selection rule.

5 Discussion

Standard aggregation underuses available coverage. The results compose into a picture that is directionally consistent across both corpora, which differ in register and punctuation profile. The oracle sits far above every real system, so the candidate set *collectively* contains the correct answer on many slots it gets wrong (Q1–Q2), and a structurally different predictor supplies most of that complementary coverage as the single largest unique contributor on both corpora (Q2). Yet the standard aggregators we test capture little of it: no standard aggregator significantly beats the best LLM on ABC, on AraPunc-sub only encoder-using systems do, and none surpasses standalone AraBERT on AraPunc-sub (Q3–Q4). The bottleneck in this regime is therefore selection, not coverage: the correct label is usually present in the candidate set, but the specific discrete-label, text-blind methods we test do not realize it from the features they are given. We do not claim aggregation is impossible in principle.

The encoder’s complementary coverage is not explained by standalone weakness. The leave-one-out analysis makes a practical point: the value of adding a predictor depends on the complementary correct coverage it supplies, not on its standalone score. Some coverage might still come from a different class bias rather than true complementarity, but on this pool a competent, genuinely complementary predictor looks more useful than another correlated LLM.

Some unused coverage is selectable, but not by one universal rule. Q4 shows the unused

coverage is not simply unreachable: a gold-label probe recovers a meaningful part of the gap, deployable router improves on the trust-the-dissenting-encoder rule in point estimate, though the AraPunc-sub gain is nonsignificant and the policy is structure- and corpus-dependent (Appendix E). The residual is concentrated in punctuation decisions with a corpus-dependent class profile rather than a single universal mark bottleneck, reinforcing that selection should condition on the *structure* of disagreement. Because our aggregators use vote, agreement, and position features but not the input Arabic words, we make no claim about a text-aware aggregator; the one text-aware mechanism we did probe, peer deliberation (Appendix F), does not break the correlation, as models overwhelmingly conform rather than re-reason independently.

6 Conclusion

We examined why aggregation yields limited gains for APR with three frontier LLMs and a fine-tuned AraBERT encoder. Across AraPunc-sub, with ABC showing the same qualitative pattern, the LLMs exhibit strongly correlated errors: when they jointly fail, they usually choose the same wrong label. Majority voting therefore has little independent evidence to exploit. At the same time, oracle analysis shows that the four-predictor pool often contains the correct label in its candidate set, and leave-one-predictor-out analysis shows that AraBERT supplies the largest unique contribution on both corpora.

The resulting diagnosis is that, in this discrete-label, vote-based, text-blind regime, the main bottleneck is selection rather than coverage. Standard aggregators recover only part of the available oracle headroom, and no tested aggregator surpasses standalone AraBERT on AraPunc-sub. Using gold labels, we find that some covered disagreements are learnable in principle, but the deployable router gives only modest, corpus-dependent gains rather than a universal selection rule.

The practical implication is therefore not simply to add more correlated LLMs, but to build committees with genuinely complementary predictors and selectors that can condition on the structure of disagreement. A finer, genre- and discourse-level account of the shared errors is beyond our slot-level analysis and remains an open direction.

7 Limitations

The study has five scope limitations. First, our main quantitative evaluation uses a deterministic sub-sample of the official AraPunc split rather than the full benchmark. We preserve split membership and the underlying token-label stream and sub-sample only for cost; this supports controlled internal comparison, although the sub-sample may not exactly match the full split’s distribution, and evaluating a sub-sample under a support-aware macro- F_1 convention means absolute scores remain non-comparable with full-benchmark results. It does not affect the internal comparisons among predictors and aggregators, which all use the same units and labels, but it remains unknown whether the measured correlation, candidate coverage, and selection patterns transfer unchanged to the official split. We therefore refer consistently to *AraPunc-sub*, avoid state-of-the-art comparisons, and release the sampled-segment identifiers, deterministic segmentation procedure, labels, and system predictions needed to reproduce the analysis.

Second, all LLM predictions use one shared zero-shot prompt. This design controls for prompt and demonstration differences, and let us study model-family variation under a common task specification, but it does not establish that the same error correlation would persist under few-shot prompting, model-specific prompt optimization, or deliberately diversified demonstrations. Such interventions may change both individual accuracy and ensemble diversity; few-shot prompting improved Arabic punctuation prediction in prior work (Al Wazrah et al., 2025), and could in principle reduce the error correlation we observe. Testing whether the correlation persists under few-shot prompting is therefore a natural next step. Our conclusions are accordingly scoped to the matched zero-shot committee evaluated here, which we view as a controlled baseline rather than the full picture of LLM aggregation for this task.

Third, the aggregation and routing methods are text-blind. They receive candidate labels, vote and agreement patterns, position, and local prediction context, but not the Arabic input tokens. This restriction is deliberate in the diagnostic analysis because it isolates whether disagreement structure alone contains sufficient information for candidate selection. It also means that our results do not show that the oracle headroom is inaccessible to a text-conditioned selector. An input-aware router

could use linguistic context to distinguish otherwise identical vote patterns and may recover additional coverage; that broader practical question is not tested here.

Fourth, ABC’s 93 test segments make it comparable in scale to AraPunc-sub and a genuine second corpus for every analysis we report; the qualitative findings, correlated errors, substantial oracle headroom, and AraBERT’s large unique contribution, hold on both. Its one limitation is statistical: the 93 segments derive from only five source documents, and because segments within a document are not independent, segment-level resampling does not increase the effective sample size beyond roughly five. We therefore rely on ABC’s wide-margin and qualitative results in full, and read only its narrow, close-to-zero significance margins with caution, deferring to AraPunc-sub where a verdict hinges on them.

Moreover, because both corpora derive from published texts, their punctuated originals may appear in the pretraining data of the LLMs. We cannot rule this out for closed models, and contamination could inflate both the models’ accuracy and their agreement. We note that our conclusions are relative comparisons among predictors on the same units, and that contamination would, if anything, reinforce rather than weaken the correlated-error finding we report.

Fifth, our pool includes only one fine-tuned Arabic encoder. Because we never pair AraBERT with a second, independently trained encoder, we cannot say whether its complementary coverage reflects something general about the encoder-LLM difference or something particular to AraBERT itself. Testing a more diverse set of Arabic encoders would help separate the two, and we leave this for future work.

References

- Asma Ali Al Wazrah, Afrah Altamimi, Hawra Aljasim, Waad Alshammari, Rawan Al-Matham, Omar El-nashar, Mohamed Amin, and Abdulrahman AIO-saimy. 2025. [Evaluation of large language models on Arabic punctuation prediction](#). In *Proceedings of the 1st Workshop on NLP for Languages Using Arabic Script*, pages 144–154, Abu Dhabi, UAE. Association for Computational Linguistics.
- Wissam Antoun, Fady Baly, and Hazem Hajj. 2020. [AraBERT: Transformer-based model for Arabic language understanding](#). In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Pro-*

- cessing Tools, with a Shared Task on Offensive Language Detection, pages 9–15, Marseille, France. European Language Resources Association.
- D. Beeferman, A. Berger, and J. Lafferty. 1998. [Cyberpunc: a lightweight punctuation annotation system for speech](#). In *Proceedings of the 1998 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP '98 (Cat. No.98CH36181)*, volume 2, pages 689–692 vol.2.
- Yves Bestgen. 2022. [Please, don't forget the difference and the confidence interval when seeking for the state-of-the-art status](#). In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 5956–5962, Marseille, France. European Language Resources Association.
- Maury Courtland, Adam Faulkner, and Gayle McElvain. 2020. [Efficient automatic punctuation restoration using bidirectional transformers with robust inference](#). In *Proceedings of the 17th International Conference on Spoken Language Translation*, pages 272–279, Online. Association for Computational Linguistics.
- Thomas G. Dietterich. 2000. Ensemble methods in machine learning. In *Multiple Classifier Systems*, pages 1–15, Berlin, Heidelberg. Springer Berlin Heidelberg.
- Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. 2018. [The hitchhiker's guide to testing statistical significance in natural language processing](#). In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392, Melbourne, Australia. Association for Computational Linguistics.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2024. [Improving factuality and reasoning in language models through multiagent debate](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 11733–11763. PMLR.
- J.G. Fiscus. 1997. [A post-processing system to yield reduced word error rates: Recognizer output voting error reduction \(rover\)](#). In *1997 IEEE Workshop on Automatic Speech Recognition and Understanding Proceedings*, pages 347–354.
- Agustin Gravano, Martin Jansche, and Michiel Bacchiani. 2009. [Restoring punctuation and capitalization in transcribed speech](#). In *2009 IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 4741–4744.
- Nizar Y Habash. 2010. *Introduction to Arabic natural language processing*. Morgan & Claypool Publishers.
- L.K. Hansen and P. Salamon. 1990. [Neural network ensembles](#). *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 12(10):993–1001.
- Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. 2023. [LLM-blender: Ensembling large language models with pairwise ranking and generative fusion](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14165–14178, Toronto, Canada. Association for Computational Linguistics.
- Philipp Koehn. 2004. [Statistical significance tests for machine translation evaluation](#). In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 388–395, Barcelona, Spain. Association for Computational Linguistics.
- Anders Krogh and Jesper Vedelsby. 1994. [Neural network ensembles, cross validation, and active learning](#). In *Advances in Neural Information Processing Systems*, volume 7. MIT Press.
- Ludmila I. Kuncheva and Christopher J. Whitaker. 2003. [Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy](#). *Mach. Learn.*, 51(2):181–207.
- Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. 2024. [Encouraging divergent thinking in large language models through multi-agent debate](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 17889–17904, Miami, Florida, USA. Association for Computational Linguistics.
- Keming Lu, Hongyi Yuan, Runji Lin, Junyang Lin, Zheng Yuan, Chang Zhou, and Jingren Zhou. 2024. [Routing to the expert: Efficient reward-guided ensemble of large language models](#). In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1964–1974.
- Evgeny Matusov, Nicola Ueffing, and Hermann Ney. 2006. [Computing consensus translation for multiple machine translation systems using enhanced hypothesis alignment](#). In *11th Conference of the European Chapter of the Association for Computational Linguistics*, pages 33–40, Trento, Italy. Association for Computational Linguistics.
- David McClosky, Eugene Charniak, and Mark Johnson. 2006. [Effective self-training for parsing](#). In *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, pages 152–159, New York City, USA. Association for Computational Linguistics.
- Sewon Min, Kenton Lee, Ming-Wei Chang, Kristina Toutanova, and Hannaneh Hajishirzi. 2021. [Joint passage ranking for diverse multi-answer retrieval](#). In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6997–7008, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. 2025. [RouteLLM: Learning to route LLMs from preference data](#). In *The Thirteenth International Conference on Learning Representations*.

Antti-Veikko Rosti, Necip Fazil Ayan, Bing Xiang, Spyros Matsoukas, Richard Schwartz, and Bonnie Dorr. 2007. [Combining outputs from multiple machine translation systems](#). In *Human Language Technologies 2007: The Conference of the North American Chapter of the Association for Computational Linguistics; Proceedings of the Main Conference*, pages 228–235, Rochester, New York. Association for Computational Linguistics.

Abdelrahman Sakr and Marwan Torki. 2023. [AraPunc: Arabic Punctuation Restoration Using Transformers](#). In *2023 20th ACS/IEEE International Conference on Computer Systems and Applications (AICCSA)*, pages 1–6. IEEE.

King Yiu Suen, Rudolf Chow, and Albert Y. S. Lam. 2025. Cantonese punctuation restoration using LLM annotated data. In *Proc. Interspeech 2025*, pages 1953–1957.

Ottokar Tilk and Tanel Alumäe. 2015. [LSTM for punctuation restoration in speech transcripts](#). In *Interspeech 2015, 16th Annual Conference of the International Speech Communication Association, Dresden, Germany, September 6-10, 2015*, pages 683–687. ISCA.

Ottokar Tilk and Tanel Alumäe. 2016. [Bidirectional recurrent neural network with attention mechanism for punctuation restoration](#). In *Proceedings of the Annual Conference of the International Speech Communication Association (Interspeech 2016)*, pages 3047–3051, San Francisco, CA, USA.

Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. 2024. [Knowledge fusion of large language models](#). In *International Conference on Learning Representations (ICLR)*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. [Self-consistency improves chain of thought reasoning in language models](#). In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

David H. Wolpert. 1992. [Stacked generalization](#). *Neural Networks*, 5(2):241–259.

Sane Yagi, Ashraf Elnagar, and Esra Yaghi. 2024a. [Arabic punctuation dataset](#). *Data in Brief*, 53:110118.

Sane Yagi, Shehdeh Fareh, Ashraf Elnagar, Mariam Balajeed, Abdalla El-mneizel, and Mohammad Al-Badawi. 2024b. [Is arabic punctuation rule-governed?](#) *Cogent Arts & Humanities*, 11(1):2303818.

Quantity	Definition / use
Macro- F_1	F_1 averaged equally over punctuation classes with nonzero gold support in the evaluated split. Primary metric; O is excluded.
Micro- F_1 (non- O)	F_1 pooled over all non- O tokens. Secondary metric; reflects performance on frequent marks.
Per-slot oracle (coverage)	Fraction of slots at which <i>at least one</i> predictor in a named pool emits the gold label. This binary quantity is distinct from oracle macro- F_1 .
Candidate-set oracle (oracle macro- F_1)	Macro- F_1 of a non-deployable system that emits the gold label on covered slots and is forced to a fixed wrong label (O , no punctuation) on uncovered slots, so each uncovered slot counts as a miss. Since macro- F_1 excludes the O class, forcing O registers as a miss on the true mark without adding a false positive to any other class. Because the forced-wrong label is a fixed choice, the absolute value depends on that choice and the metric is a diagnostic reference, not a clean upper bound; it is a strict ceiling only for aggregators constrained to <i>select</i> from the candidate labels, not for unconstrained multiclass stackers that can emit a label no predictor proposed.
LLM-only vs. four-predictor pool	The <i>LLM-only pool</i> contains the three LLMs. The <i>four-predictor pool</i> contains the three LLMs plus AraBERT. The LLM-only oracle shows what the LLM committee covers; the four-predictor oracle shows what the full predictor pool covers.
Unique contribution	Drop in the four-predictor oracle macro- F_1 when one predictor is removed; measures complementary correct coverage.
Δ macro- F_1	Difference vs. a named baseline (best LLM, best standalone, or best standard stacker), with bootstrap 95% CI ($B=1000$) over evaluation segments and one-sided p . Significant only if the CI excludes zero.

Table 8: Evaluation metrics and diagnostic quantities used in the paper.

A Evaluation Metrics and Diagnostic Quantities

Table 8 defines the evaluation metrics and oracle quantities used throughout the paper.

B Corpus Details and Statistics

AraPunc (Sakr and Torki, 2023) is distributed as a single stream in which each token is paired with its label, with no document partition. For both corpora we form evaluation segments by deterministic sentence-boundary segmentation with a maximum of 512 word tokens per segment, and sample 100 dev and 100 test segments (33,048 test to-

Property	AraPunc-sub	ABC
Register	classical	formal
Dominant mark	comma	period
Marks per 100 tokens	9.5	17.4
AraBERT (fine-tune, test F_1)	0.639	0.496
Dev segments (resampling units)	100	100
Test segments (resampling units)	100	93
Source documents (dev/test)	1/1 stream	10/5 docs
Raw test word tokens	33,048	23,884
Jointly evaluated tokens	33,048	22,176

Table 9: Corpus statistics. AraPunc-sub denotes our deterministic sub-sample of the official AraPunc split. It yields 100 resampling segments; ABC is segmented the same way and yields 93 test segments from 5 source documents.

Mark	AraPunc-sub	ABC
Comma	49.4%	36.7%
Period	17.9%	41.3%
Colon	24.9%	15.9%
Semicolon	6.4%	1.4%
Question	1.4%	2.9%
Exclamation	0.0%	1.8%

Table 10: Gold punctuation distribution (share of all non- O marks on the test split); each column is computed directly from our split and sums to 100%.

kens), to make three-frontier-LLM inference possible. For ABC, alignment uses sequence matching to recover labels when a model’s output length drifts from the input; after alignment, 93 test segments and 22,176 word slots are jointly valid across the three LLMs. Table 9 reports corpus-level statistics, and Table 10 gives the gold punctuation distribution; the two corpora differ in register, dominant mark, and punctuation density.

C Full System Comparison (Q3)

Table 11 gives the absolute macro- F_1 and the bootstrap Δ versus the best LLM (Gemini on both corpora) for every system summarized in Table 6, together with 95% confidence intervals. On ABC, all intervals include zero. On AraPunc-sub, significant gains over the best LLM are confined to systems using encoder information, and no tested aggregator surpasses standalone AraBERT.

D Per-Mark Covered-Error Analysis

Table 12 decomposes the covered-but-missed region by gold mark under two references. The raw-share columns use majority voting as the baseline and therefore track class frequency; the class-conditional columns use the best LLM and nor-

malize by the gold support of each mark. The two views use different baselines and should not be read as a single additive decomposition. After normalising by support, O is not the bottleneck (missed-coverage 3.7% on AraPunc-sub, 2.2% on ABC); the residual is concentrated in punctuation decisions.

E Disagreement-Routed Selection (Q4 Detail)

This appendix extends the Q4 summary in Section 4.5; the headline probe and router scores are in Table 7. The gold-label probe is a logistic classifier trained on the covered-disagreement subset (slots where at least one predictor is correct and at least one is wrong), with gold labels used only to define that subset; its gains over the same classifier trained on all slots establish that the candidate choice is partly learnable given a covered disagreement. Because gold defines the subset, it is diagnostic, not deployable. The router and the gold-label probe differ in which slots they act on. The router operates on *contested* slots, where the four predictors do not all agree; these are identifiable at test time from the predictions alone, with no gold, so the router is deployable. The gold-label probe operates on the narrower set of *covered-disagreement* slots, where the predictors disagree and the gold label is present in the candidate set; identifying this second condition requires the gold label, so the probe is diagnostic rather than deployable. Both learn a selector over the same features: predictor labels, agreement pattern, position, and the local prediction window. They differ only in the slots they act on. The deployable router passes unanimous slots through and routes contested slots to the learned selector, using no gold at test time. Its gain over the trust-the-dissenting-encoder rule is not significant on AraPunc-sub (bootstrap CI $[-.016, .036]$, $p=0.188$) and positive on ABC (CI $[.036, .105]$), but the ABC result rests on only five source documents, whose per-document deltas flip sign and average near zero, so we treat it as descriptive rather than conclusive.

The mechanism is corpus-dependent. On slots where the three LLMs agree and AraBERT dissents, AraBERT is correct 65.8% of the time on AraPunc-sub versus 23.3% for the unanimous LLM label, but only 43.6% versus 46.3% on ABC. AraBERT is thus a reliable dissenter on AraPunc-sub but close to a coin flip on ABC, so “al-

Corpus	System	Abs. macro- F_1	Δ vs. best LLM (95% CI)
AraPunc-sub	AraBERT standalone	0.639	+0.133 [.099, .165]
	Stacker (xgb, +AraBERT, cal)	0.569	+0.063 [.041, .086]
	Stacker (xgb, +AraBERT, raw)	0.569	+0.063 [.044, .083]
	Majority vote	0.510	+0.003
	Stacker (logistic, raw)	0.403	−0.103 [−.129, −.078]
ABC	Stacker (logistic, +AraBERT)	0.513	+0.036 [−.012, .097]
	AraBERT standalone	0.496	+0.019 [−.032, .101]
	Stacker (xgb, +AraBERT)	0.504	+0.027 [−.007, .089]
	Stacker (logistic, raw)	0.464	−0.013 [−.037, .026]
	Stacker (xgb, raw)	0.444	−0.033 [−.060, .003]

Table 11: Selected standard systems compared with the best LLM, Gemini on both corpora. On ABC, all intervals include zero. On AraPunc-sub, significant gains over the best LLM are confined to systems using encoder information, and no tested aggregator surpasses standalone AraBERT.

Gold mark	Raw share vs. majority vote		Class-cond. rate vs. best LLM	
	AraPunc-sub	ABC	AraPunc-sub	ABC
<i>O</i> (no punctuation)	68.5%	14.6%	3.7%	2.2%
Comma	18.7%	37.8%	19.1%	33.1%
Period	2.3%	38.9%	15.5%	40.2%
Colon	6.9%	5.1%	10.2%	12.7%
Semicolon	3.6%	0.6%	30.7%	13.5%
Question mark	0.1%	1.5%	0.0%	14.2%
Exclamation	—	1.4%	—	30.1%

Table 12: Two per-mark views of oracle-covered errors. Raw shares use majority voting as the reference and therefore reflect class frequency. Class-conditional rates use the best LLM as the reference and normalize by the number of gold slots of each mark. The different baselines are shown explicitly; these columns should not be read as a single additive decomposition.

ways trust the dissenting encoder” is a good policy on one corpus and a poor one on the other. A development-selected hybrid reaches 0.556 on AraPunc-sub, the best point estimate among the variants, yet still trails the encoder-feature stacker (≈ 0.569) and standalone AraBERT (0.639).

F Peer-Deliberation Probe

The main-text aggregators are text-blind. We conduct a peer-deliberation round to check whether a text-aware step, in which a model re-reads the Arabic input instead of just manipulating discrete votes, evades the correlation. Following the zero-shot pass, each model is re-prompted with the same paragraph and its peers’ predictions on the slots on which the models disagree, and asked to rework its own labels. This is a multi-agent reconsideration step, *not* an aggregator; it yields a new set of per-model predictions. For each model we report its conformity rate (on exposed disagreement slots, the proportion of times the model changes its own label to a peer’s) and the difference in macro- F_1 between its zero-shot and deliberated predictions (Table 13). A high conformity

Model	Conformity rate		macro- F_1 ($ZS \rightarrow peer$)	
	AraPunc-sub	ABC	AraPunc-sub	ABC
GPT-5.5	0.98	1.00	0.503 $\rightarrow$ 0.511	0.444 $\rightarrow$ 0.459
Claude Sonnet 4.6	0.87	0.91	0.400 $\rightarrow$ 0.491	0.388 $\rightarrow$ 0.459
Gemini 2.5 Flash	0.38	0.44	0.506 $\rightarrow$ 0.519	0.477 $\rightarrow$ 0.476

Table 13: Peer-deliberation probe. *Conformity rate* is the fraction of exposed disagreement slots on which a model abandons its own label and switches to a peer’s. *macro- F_1 ($ZS \rightarrow peer$)* gives the standalone score before (zero-shot) and after one peer-deliberation round. Exposure to disagreement induces *conformity*, not independent re-reasoning, deepening rather than breaking the error correlation of Q1.

rate means that exposure to disagreement leads to convergence rather than independent re-reasoning, which can reinforce the error correlation found in Q1. GPT-5.5 conforms on almost every one of the disputed slots, while Claude’s larger-magnitude change is likely because it is a weaker model being nudged toward stronger models, rather than because of an independent improvement. In contrast, the best individual LLM, Gemini, conforms less than half of the time, and its overall performance remains essentially flat.

G Prompt Template

All three LLMs receive the same single Arabic user message (no separate system role, for cross-provider consistency), where $\{n_words\}$ is replaced by the number of words in the paragraph. The instruction sets the allowed six-mark inventory; permits no changing, deleting, adding, re-ordering, respelling, or diacritizing of any word; and specifies that the model should leave a position unmarked if it finds no clear boundary (a bias against over-prediction). For readability, the prompt shown below is an English translation; all experiments use the original Arabic version.

“You are an expert in Arabic punctuation restoration. Rewrite the following Arabic text with exactly the same words, adding only appropriate punctuation. Do not change, delete, add, or reorder any word; do not correct spelling or add diacritics; the word count must stay exactly $\{n_words\}$. Allowed marks only: comma, semicolon, period, colon, question, exclamation. Most words take no mark; do not over-punctuate; if there is no clear pause, leave the position unmarked. Use the colon only when the preceding word directly introduces an explanation or quotation. Do not use Latin punctuation; do not output JSON; do not explain. Return only the punctuated text.”

H Reproducibility

Models and decoding. We query three frontier LLMs through their providers: gpt-5.5 (OpenAI), claude-sonnet-4-6 (Anthropic), and google/gemini-2.5-flash (Google). All calls use temperature 0 with an 8000-token limit, and Claude’s extended thinking is disabled. We do not claim bit-exact determinism, since providers do not guarantee identical outputs across backends; the run manifest records the exact version strings.

Data. For ABC we use 46 files in total. A curated stratified split of 15 files (10 for development and 5 for testing) forms the evaluation set, 30 disjoint files make up the encoder training pool, and one file is held in reserve. We release the derived labels, unit identifiers, scripts, and system predictions, but not the source text, which may remain under copyright. The split separates documents but not authors; since punctuation style is partly author-dependent, an author-disjoint split would be preferable and is left to future work. AraPunc-sub is the 100-development, 100-test segment sample described in Section 3.

Corpus	Model	Excl. rate (dev / test)
ABC	GPT-5.5	0% / 0%
	Claude	1% / 2%
	Gemini	10% / 6%
AraPunc-sub	GPT-5.5	1% / 0%
	Claude	1% / 0%
	Gemini	1% / 0%

Table 14: Per-model segment exclusion rate on each corpus, before the sequence-matching recovery pass. A segment is excluded for a model when its output does not align one-to-one with the input by word count. Rates are low overall and, on ABC, concentrated in a single model; the recovery pass resolves nearly all cases, leaving the jointly evaluated set at 93 ABC test segments and the full 100 on AraPunc-sub.

Preprocessing. We label the six focus marks together with *O*, strip decorative marks such as ellipses, guillemets, and dashes, and normalize the Arabic text. Evaluation segments are produced by a deterministic segmenter (seed 20260514) that splits at sentence boundaries and caps each segment at 512 word tokens.

Alignment. We keep a model’s output for a segment only when its words and word count match the input exactly, so that every system is scored on the same set of aligned slots. When a model’s output drifts slightly in length, a sequence-matching pass recovers the labels wherever the alignment allows, and the few segments that cannot be recovered are excluded. Word preservation before this pass was high on both corpora: on ABC it was 100% for GPT-5.5, 98% for Claude, and 94% for Gemini, and on AraPunc-sub all three models exceeded 99%.

Table 14 reports the corresponding per-model exclusion rates before the recovery pass. The rates are low across the board and, on ABC, are driven almost entirely by a single model, while the other two are at or near zero; on AraPunc-sub all three models stay at or below one percent. The recovery pass resolves nearly all of these cases, so the set we actually evaluate on, segments where all three LLM outputs parse and align, remains essentially the full test set: 93 segments on ABC and all 100 on AraPunc-sub. Because AraPunc-sub retains its complete test set, no segment is dropped there.

Encoder fine-tuning. We fine-tune AraBERT (aubmindlab/bert-base-arabertv02) as a token classifier with a batch size of 16, a maximum sequence length of 512, a warmup ratio of 0.10,

and weight decay of 0.01. For each corpus we select the learning rate and number of epochs on a 10% held-out split, searching over learning rates of 2, 3, and 5×10^{-5} and over 10 or 20 epochs, with three seeds per setting and early stopping. The selected encoder reaches 0.639 on AraPunc-sub and is stable at 0.646 ± 0.013 across three seeds. On ABC, the AraPunc default recipe underfits, so we use the validation-selected configuration (5×10^{-5} , 20 epochs), which reaches 0.4955 and is stable at 0.496 ± 0.025 across three seeds. Training uses the disjoint complement of each corpus’s split; the ABC training pool is 471 segments (112,894 tokens).

Aggregators. We choose the co-vote thresholds and select among the stackers on the development set by macro- F_1 , using document-grouped cross-validation. The XGBoost stacker uses a maximum depth of 4, 100 estimators, a minimum child weight of 10, a learning rate of 0.1, and subsampling of 0.9 for both rows and columns.

Significance. All comparisons use a paired bootstrap over evaluation segments with 1000 resamples (seed 13), from which we report the mean macro- F_1 difference, a 95% percentile interval, and a one-sided p -value, calling an effect significant only when the interval excludes zero. Because ABC’s 93 test segments come from only five documents and are not independent within a document, its effective sample size is much smaller than 93, so we read its significance results with caution.

I Prompt-Sensitivity of the Error Correlation

A natural concern is that the inter-model error correlation reported in Section 4.1 arises from using a single shared prompt rather than from the models themselves. Assigning a different prompt to each model does not provide a clean test, since it requires first identifying the best prompt per model and thereby conflates prompt quality with error decorrelation. We instead hold the prompt shared *within* each condition and vary it *across* conditions, so that any change in correlation is attributable to the prompt alone.

We evaluate the three-model committee under four prompts, applied identically to all models on a common set of AraPunc-sub test segments. The prompts differ only in their punctuation guidance,

Prompt	Mean pairwise κ
Original (mark-averse)	0.700 [.684,.717]
Neutral	0.728 [.706,.750]
Boundary-marking	0.728 [.703,.752]
Minimal (no guidance)	0.724 [.699,.749]
Main study	0.705

Table 15: Mean pairwise Cohen’s κ under four prompts applied identically to all three models on a common AraPunc-sub segment set, with 95% bootstrap intervals. The correlation is statistically indistinguishable across prompts and consistent with the main study.

ranging from our original prompt to one that removes punctuation guidance entirely. If the correlation were an artifact of our wording, it should weaken as the shared guidance is removed.

It does not. Mean pairwise Cohen’s κ is statistically indistinguishable across all four prompts (0.700–0.728; overlapping 95% bootstrap intervals) and matches the value from the main study (0.705). The other correlation signals are equally stable across the four prompts: false-comma overlap remains $16\text{--}21\times$ higher than expected under independent errors, and on slots where all three models are wrong they select the same incorrect label in 79–83% of cases, consistent with the main-study values (Table 2). Removing the guidance entirely does not reduce the correlation. The correlation is therefore a property of the model committee rather than an artifact of the shared prompt, and we scope the correlation claim to the matched zero-shot committee accordingly.

J Qualitative Examples

To make the coverage-versus-selection distinction concrete, we inspect individual test slots on AraPunc-sub (non- O slots). For each example, we show the gold-punctuated clause with the target word in brackets, followed by the mark assigned by each predictor ($\emptyset$ denotes no mark). Two patterns recur. In 340 AraPunc-sub slots, all three LLMs are wrong while AraBERT alone recovers the gold mark. In another 498 slots, no model predicts the gold mark, indicating a true coverage gap. Overall, the three LLMs agree on the same wrong label in 737 slots.

Gold-punctuated context (target in [brackets])	Gold	GPT-5.5	Claude	Gemini	AraBERT
<i>(i) The three LLMs agree on a wrong mark; AraBERT recovers it</i>					
... نص [عليه]، كسائر قتل العمد	،	∅	∅	∅	،
... أن يوصي [به]، فإن لم يوص به	،	.	.	.	،
<i>(ii) Missing coverage — no predictor proposes the gold mark</i>					
... لعدم وجوبه علي [الميت]، وسياتي بسط	.	،	،	،	،

Table 16: Qualitative examples from the AraPunc-sub test set. In case (i), the three LLMs either omit the mark or interpret a clause boundary as a period, while AraBERT recovers the gold comma. In case (ii), every predictor interprets the gold sentence boundary as a comma; therefore, the correct period is absent from the candidate set.